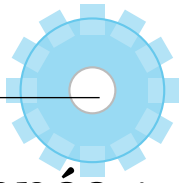
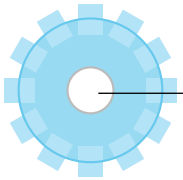


École nationale
de la statistique
et de l'analyse
de l'information



Projet Informatique de 2ème année : CleanOps



Dossier d'Analyse

17 Septembre 2026

Équipe n°34 :

MANAMBIRAVAKA Anaëlle
NADIN Sarah
NICOLIE Galahade
PERRIN-BEAU Madie
VILARRUPLA Thomas
ZHU Annabelle

Tuteur :

LACAILLE Adrien

Table des matières

Introduction **2**

1 Présentation du projet **3**

1.1 Contexte et objectifs 3

1.2 Cahier des charges 3

1.2.1 Description des fonctionnalités 3

1.2.2 Diagramme de cas d'utilisation 4

1.3 Organisation et planning 5

2 Modélisation du projet **7**

2.1 Diagramme d'activité 7

2.2 Diagramme de classes 7

2.3 Diagramme de séquence 10

2.4 Modèle physique de donnée 11

Conclusion intermédiaire **13**

Bibliographie **14**

Introduction

Aujourd'hui, le développement d'une application ne se limite plus à assurer son bon fonctionnement en s'assurant de l'absence de bugs fonctionnels. La sécurité des logiciels et de leurs dépendances constitue également un enjeu important, notamment face à la présence de vulnérabilités dans les composants utilisés. Des recommandations comme celles du National Institute of Standards and Technology (NIST) encouragent ainsi l'intégration de pratiques de sécurité tout au long du développement logiciel [2]. Par ailleurs, l'impact environnemental du numérique conduit à s'intéresser à la consommation énergétique des logiciels et à leur empreinte carbone. La méthodologie Software Carbone Intensity (SCI) propose notamment un cadre pour évaluer cet impact [1].

C'est dans ce contexte que s'inscrit CleanOps, une API REST destinée à automatiser l'audit d'un projet Python et de ses fichiers de dépendance. L'application a notamment pour objectif de vérifier l'intégrité d'une soumission, d'analyser la sécurité de ses dépendances, d'estimer son impact énergétique et carbone ainsi que de générer un inventaire des composants logiciels.

Ainsi, la problématique qui guidera notre étude est la suivante : comment concevoir une application capable d'automatiser l'audit d'un projet Python en prenant en compte à la fois sa sécurité, son impact environnemental et la traçabilité de ses composants ?

Afin d'apporter des éléments de réponse à cette question, ce rapport présente dans un premier temps le projet CleanOps dans son contexte, ses objectifs, son cahier de charges ainsi que l'organisation et le planning du groupe. La deuxième partie sera consacrée à la modélisation du projet à travers différents diagrammes. Enfin, une conclusion intermédiaire fera le bilan de cette phase d'analyse et ouvrira sur la phase de développement.

1 Présentation du projet

1.1 Contexte et objectifs

Les applications actuelles reposent massivement sur des bibliothèques open source, ce qui expose les projets à des vulnérabilités connues (appelées CVE), des licences juridiquement contraignantes ou des dépendances abandonnées. De plus, la manière dont un programme est conçu a un impact direct sur sa consommation énergétique : certains anti-patterns, comme les boucles imbriquées ou récursives mal maîtrisées, augmentent inutilement la charge de calcul et donc l'empreinte carbone associée à son exécution. L'objectif de notre application CleanOps est de concevoir une API REST capable d'auditer automatiquement un code source Python ainsi que ses fichiers de dépendances, à chaque soumission.

Concrètement, CleanOps doit permettre de :

- garantir l'intégrité et la provenance de chaque soumission grâce à une signature cryptographique
- détecter les vulnérabilités et risques liés aux dépendances (CVE, licences, packages obsolètes ou suspects, secrets exposés)
- estimer l'impact éco-énergétique du code soumis à partir de son analyse statique (AST)
- générer un inventaire standardisé des composants du projet (SBOM)
- historiser les audits et statuer, via une Quality Gate, sur la conformité d'un projet aux critères de sécurité et d'impact environnemental fixés.

CleanOps se positionne ainsi comme un outil qui aide les équipes de développement à intégrer des critères de sécurité et d'éco-responsabilité directement dans leur cycle de développement, notamment via les pipelines CI/CD.

1.2 Cahier des charges

1.2.1 Description des fonctionnalités

L'application repose sur cinq fonctionnalités principales. Chaque soumission de code est d'abord signée numériquement (HMAC), ce qui permet de vérifier qu'elle n'a pas été modifiée et qu'elle vient bien de la bonne source. Les dépendances du projet sont ensuite analysées et comparées à la base OSV pour détecter les vulnérabilités connues, les licences à risque ou les secrets exposés, ce qui permet de calculer un score de risque global. Le code est également analysé pour repérer les pratiques qui consomment inutilement de l'énergie, et cette consommation est convertie en équivalent gCO2e.

Un inventaire des composants du projet (SBOM) est généré automatiquement à chaque audit. Enfin, chaque audit est enregistré, et des seuils définis à l'avance permettent de dire si le projet respecte ou non les critères fixés. Quatre fonctionnalités supplémentaires pourraient être ajoutées selon l'avancement du projet. L'audit pourrait se lancer automatiquement dès qu'un développeur soumet du code, sans action manuelle. L'analyse

pourrait aussi s'exécuter dans un environnement isolé pour éviter qu'un code malveillant n'affecte l'application. Un assistant basé sur l'IA pourrait proposer des améliorations du code sans changer son comportement. Enfin, une interface graphique pourrait afficher la structure du projet, les dépendances, les vulnérabilités et l'empreinte carbone.

1.2.2 Diagramme de cas d'utilisation

Afin de clarifier les interactions entre l'utilisateur et la plateforme CleanOps, le diagramme de cas d'utilisation ci-dessous modélise les fonctionnalités principales accessibles depuis l'interface. L'utilisateur commence par s'authentifier auprès du système, étape préalable à toute action ultérieure. Une fois connecté il peut d'une part gérer ses projets (création, modification, ajout de composants et suppression), une opération qui repose systématiquement sur la génération d'une clé HMAC garantissant l'intégrité et l'authenticité de chaque soumission et d'autre part consulter l'historique des audits déjà réalisés.

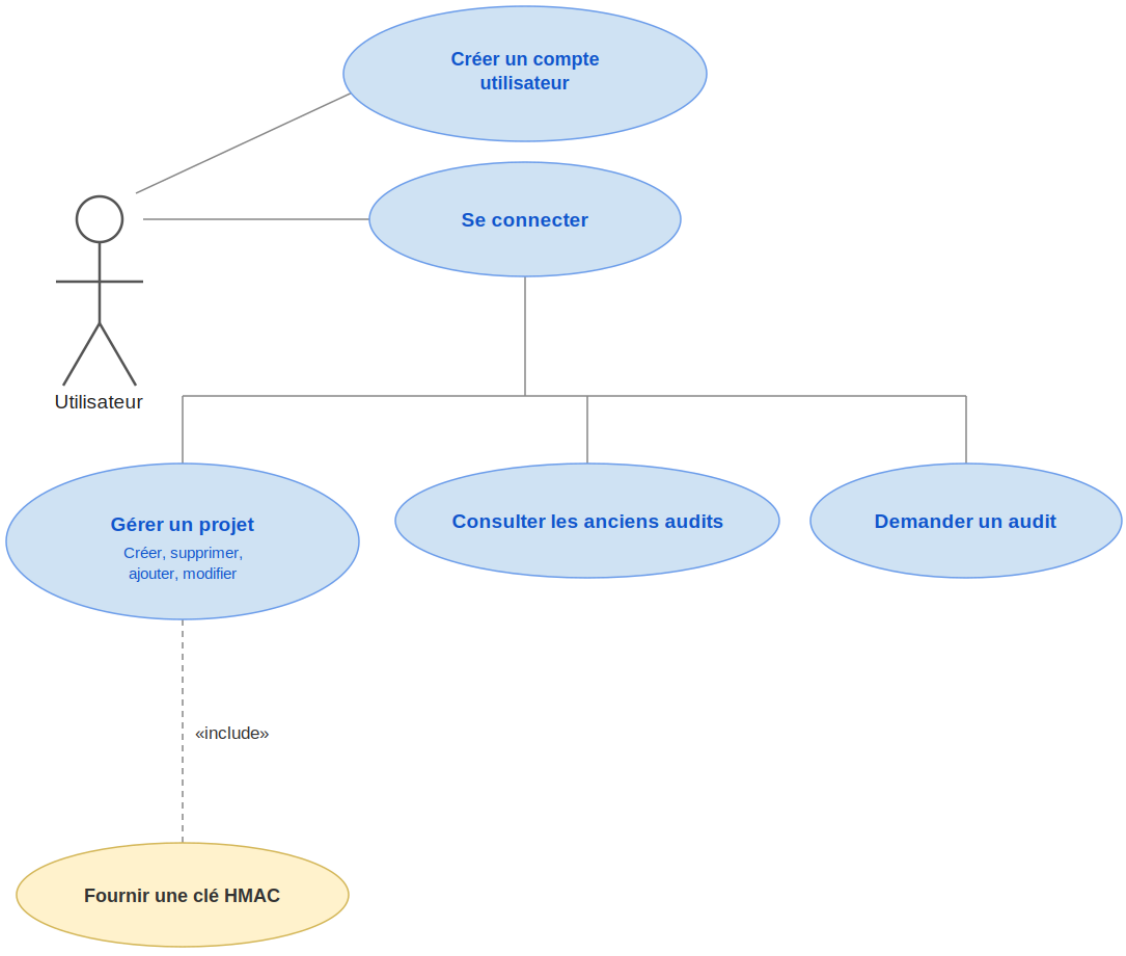


FIGURE 1 : Diagramme de cas d'utilisation

1.3 Organisation et planning

Pour organiser la réalisation du projet CleanOps, nous avons établi un diagramme de Gantt couvrant l'ensemble de la période, de début septembre à mi-décembre. Celui-ci distingue les phases d'analyse et de développement, ainsi que les dates de rendu, dont les grandes échéances sont matérialisées

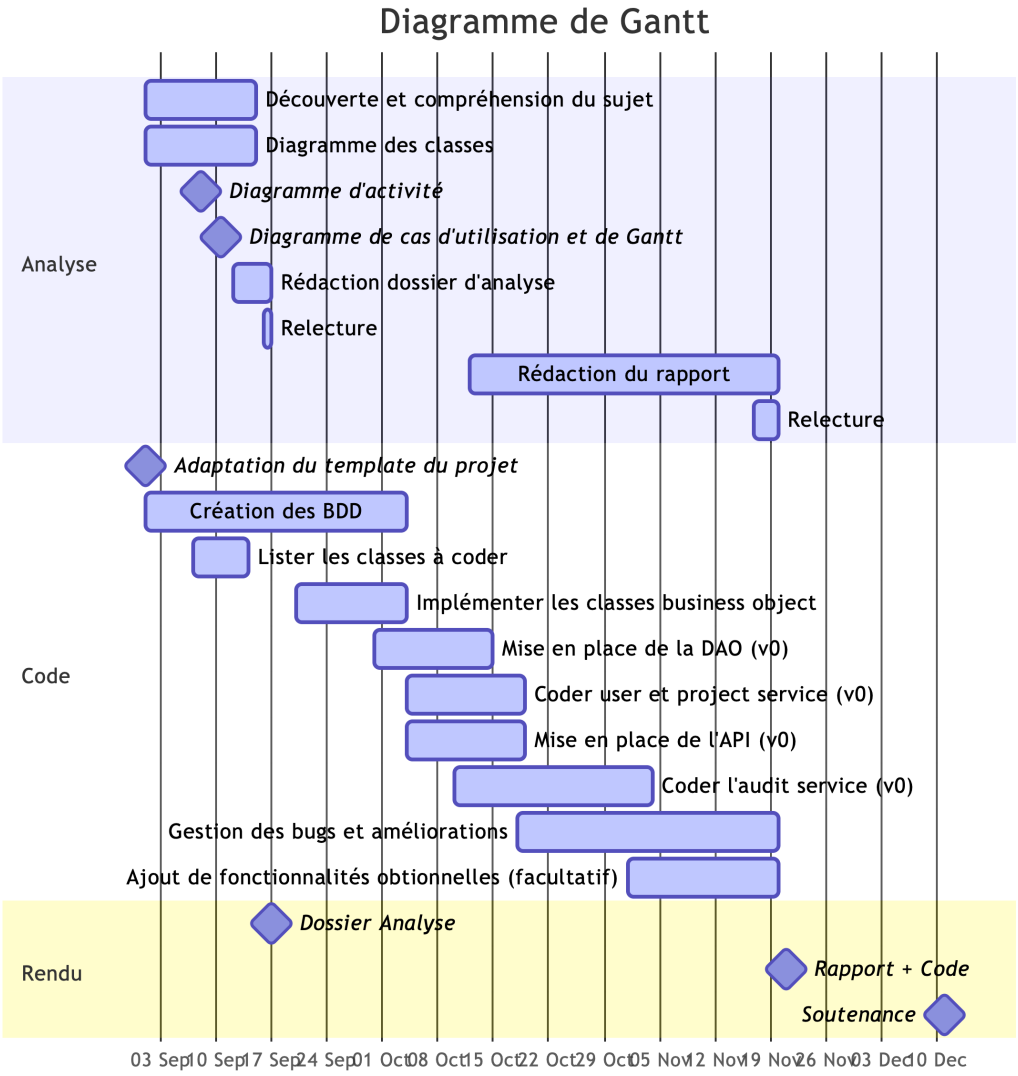


FIGURE 2 : Diagramme de Gantt

La phase d'analyse occupe les premières semaines du projet. Elle débute par la découverte du sujet et la modélisation UML (diagramme des classes, diagramme d'activité, diagramme de cas d'utilisation), menées de façon rapprochée afin de construire une compréhension commune du besoin avant de rédiger. Cette phase se conclut par la rédaction et la relecture du dossier d'analyse que voici. Une seconde phase de rédaction, celle du rapport final, est prévue plus tard, de mi-octobre à mi-novembre, une fois le développement suffisamment avancé pour en rendre compte.

La phase de développement démarre dès la fin de l'analyse, en parallèle de la rédaction du dossier. Après l'adaptation du template de projet et la création de la base de données, le travail se concentre sur l'implémentation des classes métier, puis sur le codage des fonctionnalités principales (import des fichiers et service d'audit) dans leurs versions initiales. La fin de cette phase est consacrée à la correction des bugs et à l'amélioration du code existant, en parallèle du développement des fonctionnalités optionnelles si le temps le permet.

Contrairement à ce que le diagramme de Gantt peut laisser penser au premier regard, l'analyse et le développement ne se déroulent pas vraiment en parallèle sur cette période : pour l'instant, seule l'architecture du projet a été travaillée, le code à proprement parler n'a pas encore commencé. L'essentiel du développement démarrera une fois l'analyse terminée, ce qui nous laissera les dernières semaines pour le consolider, corriger des bugs, ajouter des fonctionnalités optionnelles et rédiger le rapport final.

2 Modélisation du projet

2.1 Diagramme d'activité

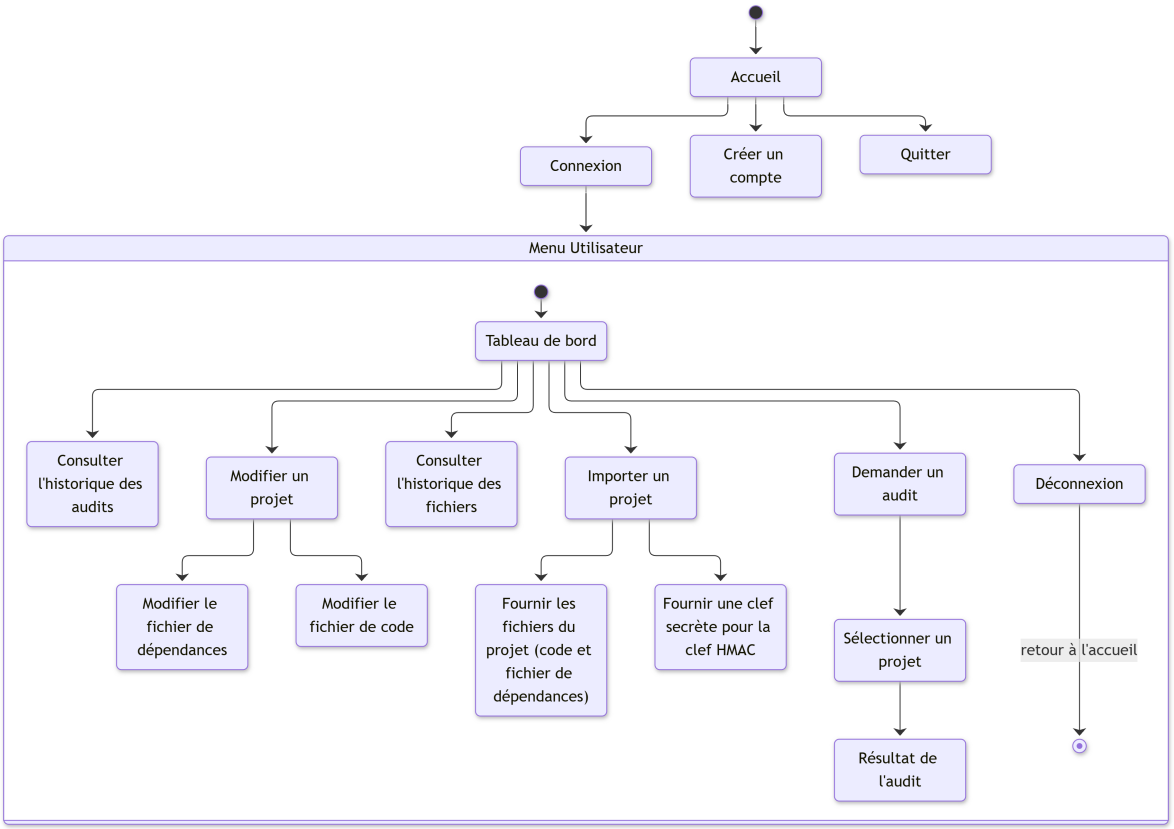


FIGURE 3 : Diagramme d'activité

2.2 Diagramme de classes

Afin de présenter l'organisation prévue de notre code de CleanOps, nous avons réalisé un diagramme de classes composé de 24 classes. Ces classes sont réparties selon leur rôle et leurs fonctionnalités, présentés dans le tableau ci-dessous :

	Principe
Objet métier	Classes représentant les principales entités manipulées par l'application.
Classe de service	Implémentation des fonctionnalités définies dans le cahier des charges, notamment la création des audits du projet.
Classe DAO	Classes qui assurent l'accès aux bases de données depuis l'application (créer, modifier, supprimer...)
Classe d'API	Assure la communication entre l'utilisateur et l'application en transmettant les requêtes entre le frontend et le backend.

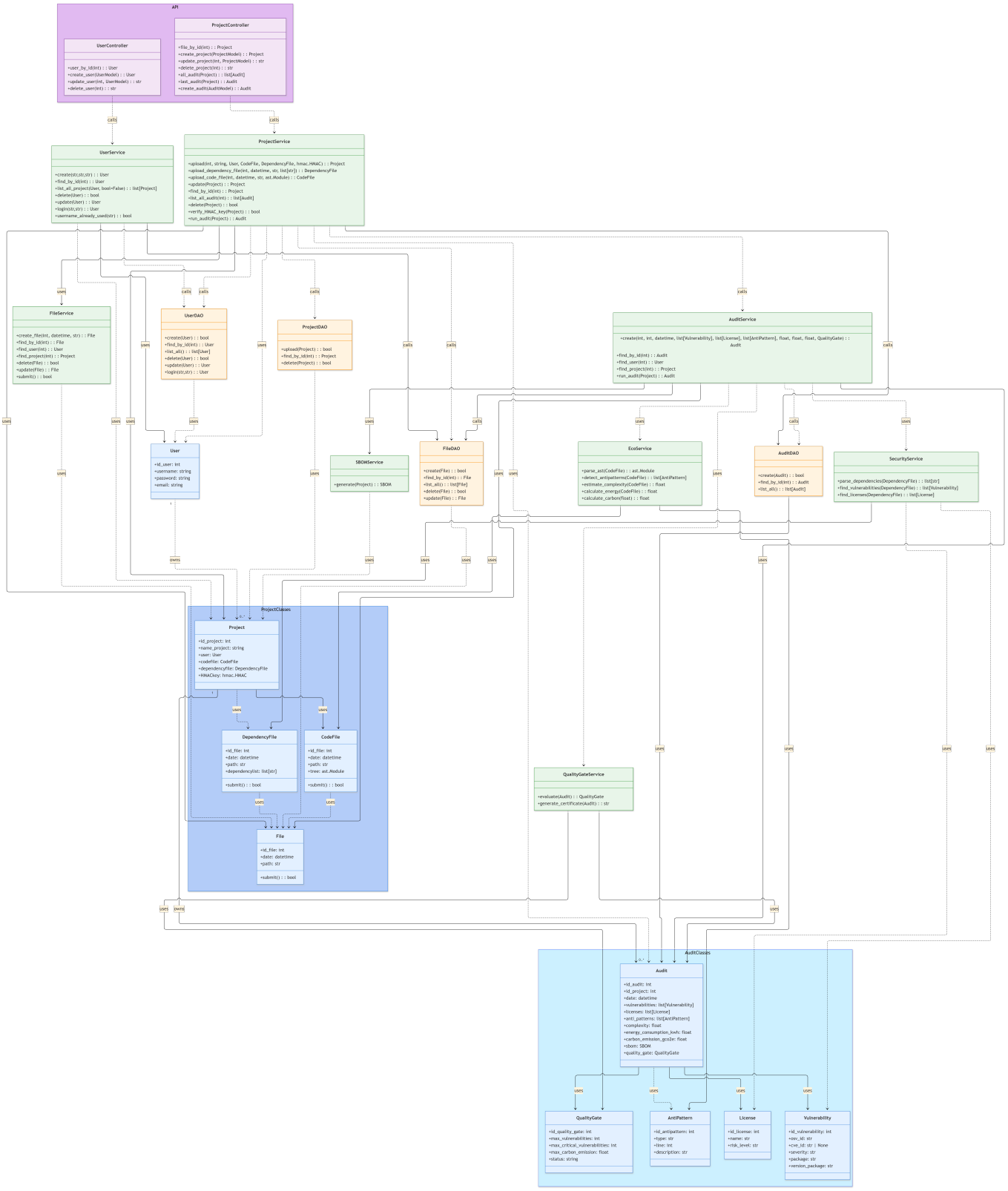


Figure 4 : Diagramme de classes

Classe User

La classe User représente un utilisateur avec ses informations d'identification (nom d'utilisateur, adresse mail et mot-de-passe). UserService assure sa gestion (création, modification, suppression, recherche et authentification) et s'appuie sur UserDAO pour l'accès aux données. UserController rend ces fonctionnalités accessibles via l'API.

Classe Projet

La classe Project représente un projet déposé par un utilisateur, et est constitué exactement de 2 fichiers : un fichier de code et un fichier de dépendances. ProjectService gère quant à lui la soumission d'un projet et de ses fichiers, la vérification de l'intégrité du projet par sa clé HMAC, la recherche d'un projet ou sa suppresion, tandis que ProjectDAO enregistre dans la base de données projet et fichiers. ProjectController assure entre autre l'accès à ces fonctionnalités via l'API.

Gestion des fichiers

La classe File constitue la classe mère de CodeFile et DependencyFile. Elle regroupe les informations communes à tous les fichiers, tandis que chaque classe fille contient les informations spécifiques à son type de fichier. CodeFile représente ainsi le fichier contenant le code source, grâce au module AST, tandis que DependencyFile représente le fichier contenant les dépendances du projet, sous forme de liste.

Réalisation des audits

Audit constitue le résultat global d'un audit réalisé sur un projet. Il regroupe les vulnérabilités, licences, anti-patterns énergétiques, complexité du code, consommation énergétique, émissions carbone, SBOM et résultat de la Quality Gate. AuditService orchestre l'ensemble des analyses grâce à plusieurs services spécialisés :

- SecurityService analyse le fichier de dépendance : recherche des vulnérabilités via notamment l'API OSV et identifie les licences, dont celles à risque
- EcoService analyse le fichier code : détecte les anti-patterns, estime sa complexité et évalue sa consommation énergétique et ses émissions carbone
- SBOMService génère l'inventaire des composants logiciels du projet
- QualityGateService compare les résultats de l'audit aux seuils définis et détermine si le projet respecte les critères de validation. Les classes Vulnerability, License, AntiPattern et QualityGate sont des objets distincts de l'Audit. Cela permet de leur associer plus d'informations si nécessaire, et de faciliter leur manipulation au sein de l'application.

Quality Gate et certificat

QualityGate définit les seuils à respecter, notamment en matière de vulnérabilités et d'émissions carbone. Lorsque les critères sont satisfaits, QualityGateService peut générer un certificat d'attestation à partir des informations de l'audit.

2.3 Diagramme de séquence

Comme nous pouvons le voir plus précisément avec le diagramme de séquence, l'architecture de notre application est organisée autour de plusieurs couches. Les classes Controller constituent le point d'entrée de l'application et permettent de recevoir les requêtes des utilisateurs, comme créer un compte ou importer un projet. Elles communiquent avec les classes Service, qui appliquent la requête de l'utilisateur. Ces services utilisent ensuite les classes DAO lorsqu'une interaction avec les bases de données est nécessaire.

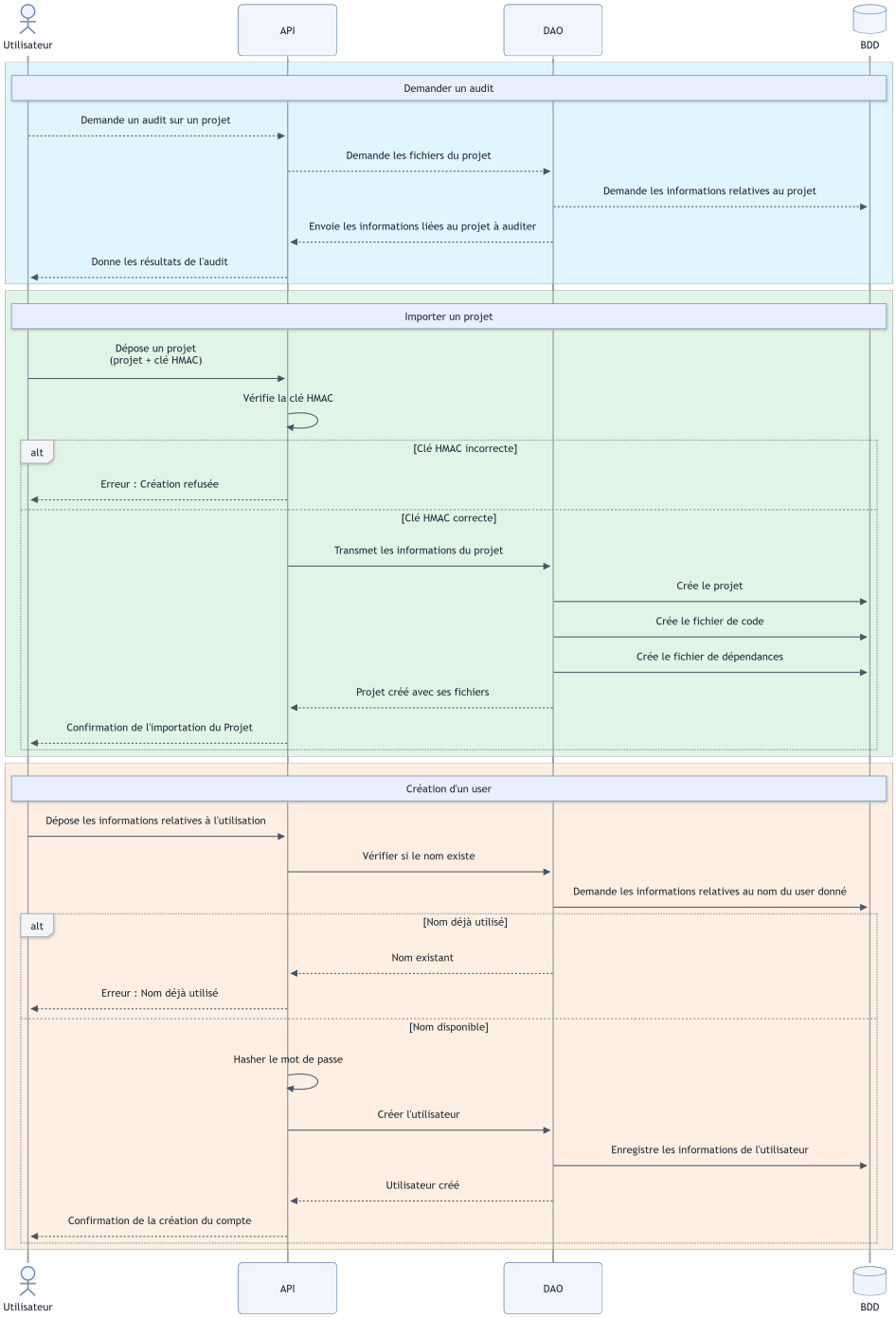


Figure 5 : Diagramme de séquence

2.4 Modèle physique de donnée

Ce modèle physique de données décrit la structure de la base de données de l'application. Il comporte 4 tables : User, Project, File et Audit.

User : La table User contient les données liées aux utilisateurs de l'application

- id_user : identifiant unique de l'utilisateur (clé primaire)
- username : chaîne de caractères ne dépassant pas 50 caractères qui est le nom de l'utilisateur
- password : mot de passe de l'utilisateur (100 caractères max)
- email : adresse mail

Chaque utilisateur peut posséder 0, 1 ou plusieurs projets. Un utilisateur possède 0 projet lorsqu'il a créé un compte, mais n'a encore importé aucun projet, ou lorsqu'il a supprimé les projets qu'il avait précédemment importés.

Project : La table Project représente un projet soumis à l'audit par un User.

- id_project : identifiant unique du projet (clé primaire)
- name_project : nom du projet
- id_user : clé étrangère qui réfère à l'utilisateur qui a soumis le projet
- HMACkey : clé HMAC utilisée pour garantir la provenance et l'intégrité du projet

Un projet appartient à un seul utilisateur et contient exactement deux fichiers (le code et le fichier de dépendances). Un projet peut être soumis à 0, 1 ou plusieurs audit.

File : La table File représente les fichiers importés.

- id_file : identifiant unique du fichier (clé primaire)
- name_file : nom du fichier
- id_project : clé étrangère qui réfère au projet dont le fichier appartient
- type_file : type du fichier (code ou fichier de dépendances)
- date : date d'ajout du fichier (ou dernier mise à jour du fichier)

Chaque projet contient exactement deux fichiers, un code et un fichier de dépendances.

Audit : La table Audit représente le résultat d'un audit effectué sur un projet.

- id_audit : identifiant unique de l'audit (clé primaire)
- id_project : référence vers le projet audité (clé étrangère)
- date : date à laquelle l'audit a été réalisé
- complexity : indicateur de complexité du code analysé
- energy_consumption_kwh : estimation de la consommation énergétique du code
- carbon_emission_gco2e : estimation des émissions carbone associées
- vulnerabilities : liste des vulnérabilités détectées dans le code ou les dépendances
- sbom : liste des composants et dépendances du projet (packages, leurs versions, licence, etc.)

Un projet peut posséder 0 à plusieurs audits sachant qu'un audit doit obligatoirement être lié à un projet.

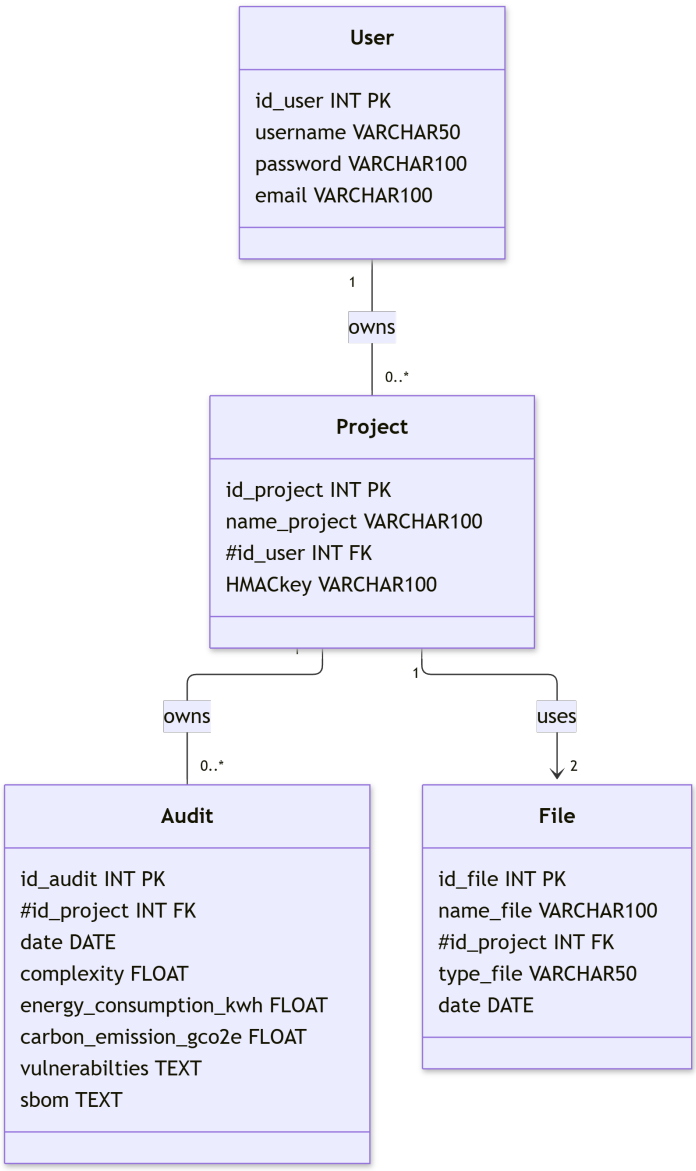


FIGURE 6 : Diagramme physique de donnée

Conclusion intermédiaire

En guise de conclusion, cette première phase du projet a permis de mieux comprendre les enjeux et les objectifs de CleanOps et de préciser les différentes fonctionnalités attendues. L'étude du cahier des charges a amené à réfléchir aux besoins de l'application, notamment en termes de sécurité des dépendances, d'analyse du code, d'estimation de son impact environnemental et de traçabilité des composants logiciels ? Cette étape a également permis d'identifier les principaux acteurs et composants intervenant dans le fonctionnement de l'application.

La modélisation a ensuite permis de traduire ces besoins en une conception plus concrète du projet. De plus, les diagrammes réalisés ainsi que le modèle physique des données ont aidé à représenter le fonctionnement de l'application et les relations entre ses différents éléments. Par ailleurs, les recherches nécessaires à la conception de CleanOps ont permis de nous familiariser avec différentes technologies et références liées à la sécurité et à l'analyse logicielle.

Cette phase d'analyse constitue ainsi une base de travail commune avant le passage à la réalisation. D'ailleurs, la phase de développement aura pour objectif de mettre progressivement en uvre les choix effectués dans le cahier des charges. Elle permettra également de confronter la conception aux contraintes réelles de l'implémentation. Cette transition entre l'analyse et la réalisation permettra ainsi de vérifier concrètement la cohérence et la faisabilité des choix effectués au cours de cette première phase.

Bibliographie

[1] Green Software Foundation. (2024). *Software Carbon Intensity (SCI) Specification*. Green Software Foundation. Consulté le 16 septembre 2026. https://sci.greensoftware.foundation/

[2] National Institute of Standards and Technology. (2022, 3 février). *Secure Software Development Framework (SSDF) Version 1.1 : Recommendations for Mitigating the Risk of Software Vulnerabilities*. NIST Special Publication 800-218. https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-218.pdf